1. 개발 환경 구성

가. 설치 서버

구분	서버명	사용 포트
DB	PostgreSQL + PostGIS	5432
Graph DB	Virtuoso	8890, 1111
Graph DB	Neo4j	7474, 7687
Search Engine	Elasticsearch + Kibana	5601, 9200
App	Platform Admin API	6998
App	Metadata API	16997
App	Metadata Web	8084
App	인증 서버	33000

나. DB 설치 (PostgreSQL + PostGIS)

- PostgreSQL 버전 12 이상 설치

https://www.postgresql.org/download/ 참고

- PostGIS 설치

ex) sudo apt install postgis postgresql-postgis

- Platform Admin API 용 과 Metadata API 용 DATABASE 와 USER 를 따로 생성해도 됨.
  여기서는 1개만 생성함.

- psql 실행

1. DATABASE 생성

```
postgres=# create database dtmeta encoding 'utf-8';
```

2. USER 생성, 권한 부여, DATABASE 소유자 변경

```
postgres=# create user meta_manager password 'rhksflwk1!';
postgres=# alter user meta_manager createdb;
postgres=# alter database dtmeta owner to meta_manager;
```

3. dtmeta DATABASE 에 SCHEMA 생성

```
postgres=# \c dtmeta
dtmeta=# create schema dtmeta authorization meta_manager;
dtmeta=# create schema common authorization meta_manager;
```

4. dtmeta SCHEMA 에 PostGIS 확장

```
dtmeta=# create extension postgis with schema dtmeta;
```

- DDL, DML 실행

1. DBeaver 와 같은 Tool 을 사용 하여 DB 접속.

```
url: jdbc:postgresql://{Server IP}:5432/dtmeta
username: meta_manager
password: rhksflwk1!
```

2. 아래 파일의 SQL 문 실행

ddl_common_schema.sql
ddl_dtmeta_schema.sql
dml_common_schema.sql
dml_dtmeta_schema.sql

- Platform Admin API / Metadata API application-env.yml 설정

```
datasource:
  url: jdbc:postgresql://[redacted]:5432/dtmeta
  username: meta_manager
  password: rhksflwk1!
  driver-class-name: org.postgresql.Driver
```

다. Virtuoso 설치

• Docker run

```
sudo docker run \
  --name virtuoso \
  --publish=1111:1111 \
  --publish=8890:8890 \
  --volume=/usr/local/application/virtuoso/database:/database \
  --detach \
  -e DBA_PASSWORD=rhksflwk1! \
  openlink/virtuoso-opensource-7:7.2.6-r10-g5a27710
```

• Virtuoso Conductor 접속

ex) http://[Server IP]:8890/conductor



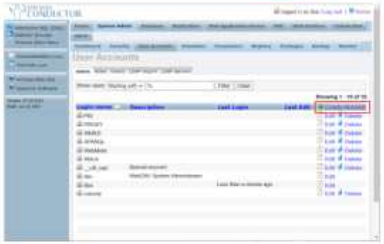
• Virtuoso Conductor 설정

1. dba 계정 로그인



Account: dba / Password: rhksflwk1!

2. System Admin – User Accounts 메뉴에서 meta_manager 계정 생성





Account Name: meta_manager

User Login: meta_manager

Password: rhksflwk1!

User Type: SQL/ODBC

Primary Role: dba

Account Role: administrators

SPARQL_LOAD_SERVICE_DATA

SPARQL_SELECT

SPARQL_SPONGE

SPARQL_UPDATE

Default Permissions: 전부 체크

입력 후 save 버튼 클릭

3. dba 계정 로그아웃 후 meta_manager 계정 로그인

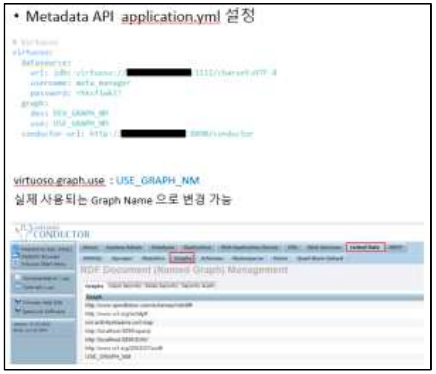


Account: meta_manager / Password: rhksflwk1!

4. Linked Data – SPARQL 메뉴에서 쿼리 정상 동작 확인



• Metadata API application.yml 설정



virtuoso.graph.use : USE_GRAPH_NM

실제 사용되는 Graph Name 으로 변경 가능

라. Neo4j 설치

• Docker run

```
sudo docker run \
  --name neo4j \
  --publish=7474:7474
  --publish=7687:7687 \
  --volume=/usr/local/application/neo4j/data:/data \
  --volume=/usr/local/application/neo4j/conf:/conf \
  --volume=/usr/local/application/neo4j/plugins:/plugins \
  --env NEO4JLABS_PLUGINS='["apoc", "n10s"]' \
  --env NEO4J_AUTH=neo4j/rhksflwk1! \
  --env DBMS_SECURITY_AUTH_ENABLED=true \
  --env DBMS_UNMANAGED_EXTENSION_CLASSES=n10s.endpoint=/rdf \
  --detach \
  neo4j:latest
```

• Neo4j Browser 접속 및 설정

ex) http://(Server IP):7474/browser



1. 로그인



Username: neo4j / Password: rhksflwk1!

2. 초기 설정

1) graph config 초기 설정

CALL n10s.graphconfig.init()



2) 제약 사항 설정

(Resource에 저장할 때 uri가 중복되지 않도록 제약)



```
CREATE CONSTRAINT n10s_unique_uri FOR (n:Resource)
REQUIRE n.uri IS UNIQUE
```

3) 제약 사항 설정

(medataResource에 저장할 때 medataSetId가 중복되지 않도록 제약)



```
CREATE CONSTRAINT FOR (n:medataResource)
REQUIRE n.medataSetId IS UNIQUE
```

• Metadata API application.yml 설정

```
# NEO4J
neo4j:
  authentication:
    username: neo4j
    password: rhksflwk1!
  uri: bolt://[redacted]:7687
  browser-url: http://[redacted]:7474/browser
```

마. Elasticsearch & Kibana 설치

• Git clone

https://github.com/deviantony/docker-elk.git

• docker-compose

cd /docker-elk

docker-compose up setup -d

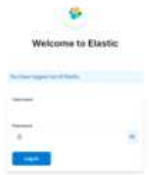
docker-compose up –d

• kibana 접속

ex) http://[Server IP]:5601



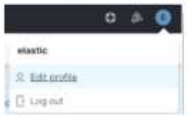
1. 로그인



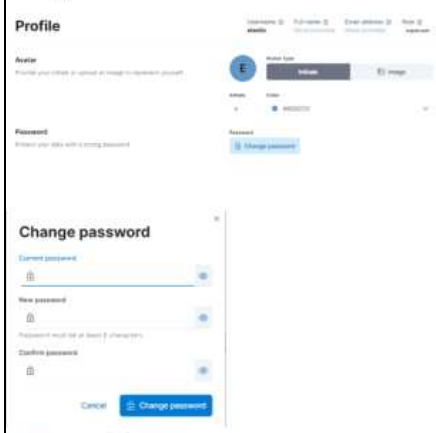
Username: elastic / Password: changeme

2. 패스워드 변경

우측 상단 프로필 – Edit profile 클릭



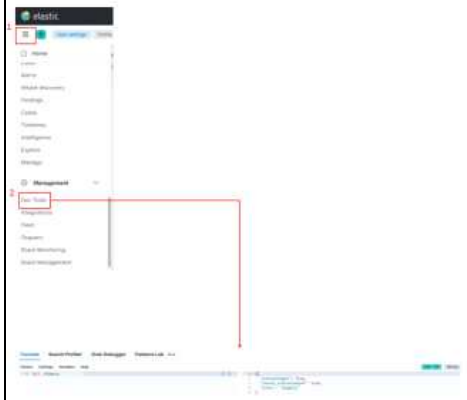
3. Change password 버튼 클릭 후 비밀번호 변경



Currnet password : changeme

New password : rhksflwk1!

4. 인덱스 추가



PUT /dtmeta

• Metadata API application.yml 설정



2. 구동/실행

가. Platform Admin API 설치

• Java 설치 (버전 11이상)
https://download.eclipse.org/justj/jres/11/downloads/latest/
ex) org.eclipse.justj.openjdk.hotspot.jre.full-11.0.18-win32-x86_64.tar.gz

• Maven 설치 (버전 3.9 이상)
https://maven.apache.org/download.cgi
ex) apache-maven-3.9.5-bin.tar.gz

• Java Tool 설치
ex) 이클립스 https://www.eclipse.org/downloads/packages/release/2022-12/r
STS4 등

• Git clone
설치 경로에서
git init
git remote add -f origin https://etrioss.kr/fdtfw_grp/fdtfw.git
git config core.sparseCheckout true
echo "CDFW/keti_dt_platform_admin_ms" >> .git/info/sparse-checkout
git pull origin CDFW_development_n2m

ex)

• Spring boot 실행
context-path: /admin
local
keti_dt_platform_admin_ms [####]

ex) console

나. Metadata API 설치

• Java 설치 (버전 11이상)
https://download.eclipse.org/justj/jres/11/downloads/latest/
ex) org.eclipse.justj.openjdk.hotspot.jre.full-11.0.18-win32-x86_64.tar.gz

• Maven 설치 (버전 3.9 이상)
https://maven.apache.org/download.cgi
ex) apache-maven-3.9.5-bin.tar.gz

• Java Tool 설치
ex) 이클립스 https://www.eclipse.org/downloads/packages/release/2022-12/r
STS4 등

• Git clone
설치 경로에서
git init
git remote add -f origin https://etrioss.kr/fdtfw_grp/fdtfw.git
git config core.sparseCheckout true
echo "CDFW/keti_dt_metadata_api" >> .git/info/sparse-checkout
git pull origin CDFW_development_n2m

ex)

• Spring boot 실행
context-path: /meta
local
keti_dt_metadata_api [16997]

ex) console



다. Metadata Web Frontend 설치

• npm 설치 (nodejs 버전 18.16.1 이상)
https://nodejs.org/en/download 참고

• vuejs3 를 사용하기 위한 Tool 설치
ex) VSCode https://code.visualstudio.com/download 등

• Git clone
설치 경로에서
git init
git remote add -f origin https://etrioss.kr/fdtfw_grp/fdtfw.git
git config core.sparseCheckout true
echo "CDFW/keti_dt_metadata_web" >> .git/info/sparse-checkout
git pull origin CDFW_development_n2m

ex)

• vuejs3 실행

ex) VSCode 터미널 창

> npm install

> npm run local

ex)



• {Server IP}:8084 로그인

(2개의 API 서버, 인증 서버 먼저 실행해야 함)

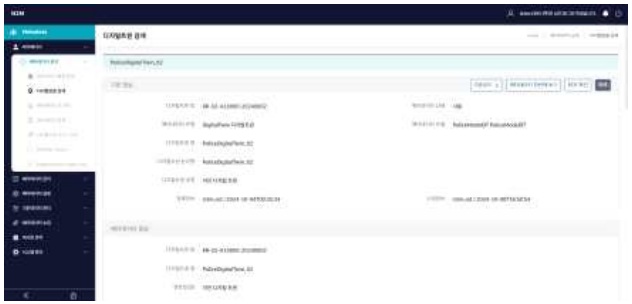


아이디: meta_admin

비밀번호: rhksflwk1!

라. Metadata Web 실행

- 디지털 트윈 검색



- 디지털 트윈 메타데이터 관리



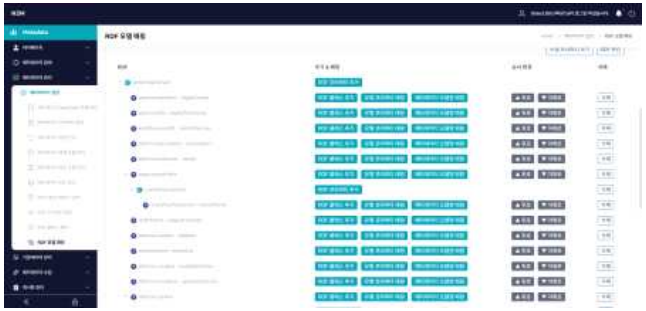
- 디지털 트윈 메타데이터 대상 SPARQL 질의 지원



- 디지털 트윈 메타데이터 데이터 모델 관리



- 디지털 트윈 메타데이터 RDF 변환



- 디지털 트윈 메타데이터 시각화 기능

